YEAR 12 · AI ENGINEERING · LECTURE 2

LangGraph

Building AI workflows that can branch, loop, remember and pause

A chain is a train line. A graph is a metro map.

TODAY

1. Why chains aren't enough
2. State, nodes, edges
3. Your first graph
4. Conditional edges and loops
5. Memory with checkpointers
6. Agents are graphs
7. Multi-agent patterns
8. Activity + quiz

When a straight line isn't enough

- A LangChain chain runs **A → B → C**, always in the same order
- Real tasks need decisions:
  - **If** the answer looks wrong, try again
  - **If** it's a maths question, use the calculator; **otherwise** search the notes
  - **Loop** until the essay scores at least 8/10 (but at most 3 times!)
  - **Pause** and ask a human before sending an email (Lecture 4)
- **LangGraph** (from the LangChain team) builds these workflows as **graphs**
- Used for **agents** and long-running, reliable AI workflows in industry

TALKING POINTS

Draw a single train line on the board (A-B-C), then a metro map with junctions and a circle line. Ask: which one describes how a person actually does homework? (You check your answer, go back, look things up — branches and loops.) That's why we need graphs.

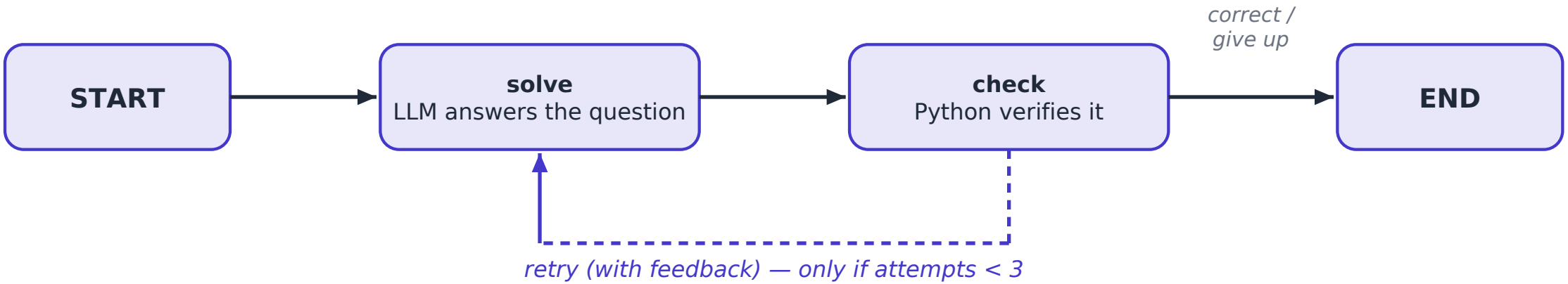
The four core ideas

Idea	What it is	Metro-map analogy
State	A shared notebook (a Python dict / TypedDict) that every step can read and update	Your ticket, stamped at each station
Node	A Python function: takes the state, returns updates. Often calls an LLM or a tool	A station where something happens
Edge	Which node runs next	A track between stations
Conditional edge	A function that looks at the state and picks the next node	A junction with signals
START / END	Special entry and exit points	Where you get on and off

TALKING POINTS

Spend time on State — it's the idea students find most new. Every node gets the whole state and returns only the parts it changed; LangGraph merges the updates. This makes each node small, testable and independent.

Your first graph: a homework checker



State = { question, expected, answer, attempts, feedback }

The **conditional edge** after *check* reads the state and returns a label: "correct", "retry" or "give up".

TALKING POINTS

Trace one run aloud: START → solve writes an answer into the state → check compares it with the expected answer → the router reads the state and returns 'correct', 'retry' or 'give up'. Ask: what would happen without the attempts limit if the model kept getting it wrong? (An infinite loop that costs money forever.) Every loop needs a stopping rule.

The homework checker in code (1/2): state and nodes


```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class HomeworkState(TypedDict):
    question: str
    expected: str
    answer: str
    attempts: int
    feedback: str

def solve(state):
    hint = f"Your last answer was wrong: {state['feedback']}. " if state["feedback"] else ""
    reply = llm.invoke(hint + state["question"] + " Give only the number.").content
    return {"answer": reply, "attempts": state["attempts"] + 1}   # only what changed

def check(state):
    ok = state["expected"] in state["answer"].replace(",", "")
    return {"feedback": "" if ok else f"you said {state['answer']}"}
```


TALKING POINTS

Point out that nodes are ordinary Python functions. solve returns only answer and attempts; check returns only feedback. LangGraph merges these updates into the state. Students can unit-test each node on its own with a hand-made state dict.

The homework checker in code (2/2): edges and running it


```
def route(state):
    if state["feedback"] == "":
        return "correct"
    return "give up" if state["attempts"] >= 3 else "retry"

g = StateGraph(HomeworkState)
g.add_node("solve", solve)
g.add_node("check", check)
g.add_edge(START, "solve")
g.add_edge("solve", "check")
g.add_conditional_edges("check", route, {"correct": END, "retry": "solve", "give up": END})
app = g.compile()

result = app.invoke({"question": "What is 48 x 25 - 17?", "expected": "1183",
                     "answer": "", "attempts": 0, "feedback": ""})
print(result["attempts"], result["answer"])
print(app.get_graph().draw_mermaid())       # draw the graph!
```


TALKING POINTS

Read the add_conditional_edges line slowly: from 'check', call route(), then follow the label to the next node. compile() checks the graph is valid and returns an app with the same invoke/stream methods as LangChain chains. draw_mermaid() gives a diagram you can paste into mermaid.live — great for checking students' graphs.

Memory with checkpointers

- A **checkpointer** saves the state after **every** step
- Each conversation is a **thread** with its own `thread_id`
- This gives you:
  - **Memory** — continue a conversation later, on the same thread
  - **Time travel** — look at (or replay from) any earlier step
  - **Pause and resume** — stop for a human, continue hours later (Lecture 4)
  - **Fault tolerance** — if something crashes, restart from the last checkpoint
- `app = g.compile(checkpointer=InMemorySaver())` — in production use a database (e.g. Postgres)
- `app.invoke(inputs, {"configurable": {"thread_id": "aroha-chat-1"}})`

TALKING POINTS

Analogy: a video game save point. If you die, you go back to the last save instead of the start; and you can have several save files (threads) for different players. In-memory savers disappear when the program stops — real apps store checkpoints in a database.

A chatbot with memory in LangGraph


```
from langgraph.graph import MessagesState
from langgraph.checkpoint.memory import InMemorySaver

def chatbot(state: MessagesState):
    return {"messages": [llm.invoke(state["messages"])]}   # appended, not replaced

g = StateGraph(MessagesState)
g.add_node("chatbot", chatbot)
g.add_edge(START, "chatbot"); g.add_edge("chatbot", END)
buddy = g.compile(checkpointer=InMemorySaver())

rua = {"configurable": {"thread_id": "rua"}}
buddy.invoke({"messages": [("user", "Hi, I'm Rua. I love chemistry.")]}, rua)
buddy.invoke({"messages": [("user", "What subject do I love?")]}, rua)    # remembers

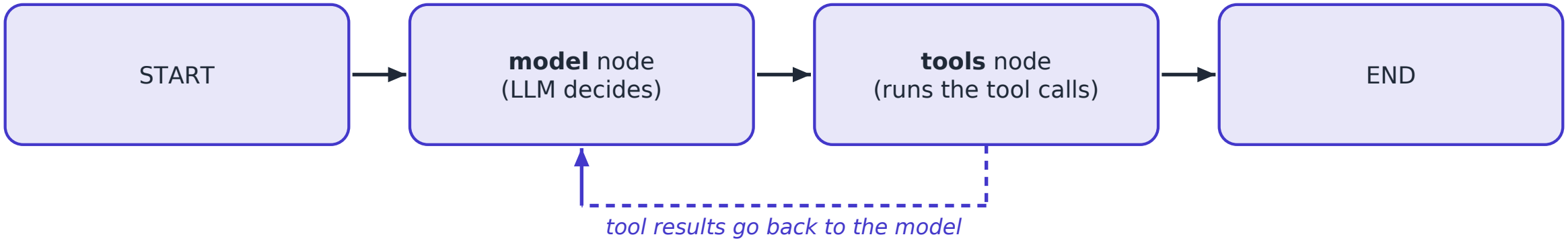
other = {"configurable": {"thread_id": "someone-else"}}
buddy.invoke({"messages": [("user", "What subject do I love?")]}, other)  # doesn't
```


TALKING POINTS

MessagesState is a ready-made state with a messages list; new messages are appended. Compare with Lecture 1's RunnableWithMessageHistory — same idea, but now memory is a feature of the whole graph, so it works for complex workflows too.

Agents are graphs too

The **ReAct agent** loop is just a two-node graph with a conditional edge: "did the model ask for a tool?"



LangChain's `create_agent(...)` from Lecture 1 builds exactly this graph for you. With LangGraph you can add your own nodes: a safety check before tools, a reviewer after the answer, a human approval step...

TALKING POINTS

Big reveal: the agent from last lecture is a LangGraph graph underneath. The conditional edge after the model node asks: does the last message contain tool calls? If yes → tools node, then back to the model. If no → END. Once students see this, agents stop being magic.

Multi-agent patterns

Pattern	How it works	School example
Router	A first node classifies the request and sends it down one branch	Maths → calculator branch; rules → handbook branch
Supervisor	A manager agent hands sub-tasks to specialist agents and combines results	Manager + researcher + writer + quiz-maker
Reflection	One node drafts, another critiques, loop until good enough	Essay writer + strict marker
Plan-and-execute	Plan all steps first, then run them one by one	Organise a class trip step by step

TALKING POINTS

Each pattern is just a different graph shape. Ask students which pattern fits: 'a help desk that answers about sports, timetables and wellbeing' (router or supervisor), 'write a speech and improve it' (reflection).

LangChain vs LangGraph

	LangChain	LangGraph
Shape	Chains: mostly straight lines	Graphs: branches, loops, parallel steps
Control	Quick to build with defaults	Exact control over every step and decision
State	Passed along the chain	Shared state object + checkpoints
Long tasks	Not designed for pausing	Pause, resume, human approval, retries
Use together?	Yes — models, prompts, tools, retrievers…	…wired up as nodes in a LangGraph graph

TALKING POINTS

Stress that it's not either/or: LangGraph nodes usually use LangChain pieces inside. Think of LangChain as the parts and LangGraph as the wiring diagram.

Activity — draw a graph (12 minutes, groups of 3)

- Design a **School Help Desk** graph on paper:
  - Questions about **school rules** → search the handbook
  - **Maths** questions → a calculator tool
  - Anything about **feeling unwell or upset** → a kind reply + tell them which staff member to see
  - Before answering, a **checker** node makes sure the reply is polite; retry at most 2 times
- Write down: the **state** fields, every **node**, every **edge**, and each **router** function's labels
- Bonus: where would you add a **checkpointer**, and why?

TALKING POINTS

Give groups A3 paper. Look for: a router node at the start, a loop with a counter in the state, and clear END points. Pick one group to present; translate their diagram into add_node / add_edge calls together on the board.

Recap + exit quiz

- LangGraph = **state** + **nodes** (functions) + **edges**; **conditional edges** make branches and loops
- Nodes return only what they **change**; LangGraph merges it into the state
- **Checkpointers** save every step per **thread** → memory, resume, time travel
- Agents are graphs: model ⇄ tools; multi-agent systems are bigger graphs
- **Quiz:** 1) What does a router function return? 2) Why must every loop have a limit? 3) What is a thread_id for? 4) Which LangGraph feature lets a workflow pause for a human?

TALKING POINTS

Answers: 1) A label (string) naming the next node. 2) Otherwise it might run forever and cost money. 3) Keeping separate conversations / saved states. 4) Checkpointers together with interrupts — the topic of Lecture 4. Next: Lecture 3, RAG — giving models an open book.